

Claude Code: Latest Features & Improvements

A curated update for a senior .NET / Blazor developer — covering the Claude Code Plugin Marketplace, on-prem Azure DevOps CI/CD, and daily IDE workflow · September 2026

Claude Code ships very frequent releases (multiple times a week). Rather than listing every changelog line, this document picks out the additions and changes most likely to matter for your work: building and maintaining your internal Claude Code Plugin Marketplace, running Claude Code against your on-prem Azure DevOps pipelines, and day-to-day .NET/Blazor development in VS Code on Windows. Each item is described in up to three sentences.

1. Plugin Development & Your Internal Marketplace

claude plugin eval

A new **claude plugin eval** command runs a plugin's test suite against Claude Code and returns scored, reproducible results as JSON and HTML reports. This gives you a repeatable way to quality-check plugins before they go into your internal Claude Code Plugin Marketplace. It can be wired into an Azure DevOps pipeline stage as an automated gate.

claude plugin validate --json

claude plugin validate now supports a **--json** flag, producing a machine-readable validation report instead of plain text. This makes it straightforward to fail a CI build automatically when a plugin doesn't pass validation. It also closes several path-traversal and symlink loopholes that previously let a plugin read files outside its own folder.

--plugin-dir now loads whole folders of plugins

You can point **--plugin-dir** at a folder containing several plugin subfolders, and every child with a manifest loads automatically. Plugins added or removed while Claude Code is running are picked up live. This speeds up local development when you're maintaining multiple marketplace plugins side by side.

Instant plugin management + --json output

claude plugin install / uninstall / update / enable / disable now accept **--json** for scripting, and enabling or disabling a plugin from **/plugin** takes effect immediately in open sessions — **/reload-plugins** is no longer needed. This shortens the edit-test loop while iterating on marketplace plugins.

/skill-doctor

The new **/skill-doctor** command lists which loaded skills are going unused and how much context budget each one costs. Handy for pruning your marketplace's skills so they don't bloat every developer's context window unnecessarily.

2. IDE & Daily Workflow (VS Code)

Live diff panel (/diff)

Fullscreen mode now has a diff panel that sits beside the conversation and updates live as Claude edits your files, toggled with **/diff**. You can review C#/Blazor changes as they happen without switching to git or your IDE's own diff view. It also now opens fully rendered in one step instead of flashing a loading state.

VS Code: Agent map for sub-agents

An "N agents" footer pill in VS Code opens a map of a session's sub-agents, each with its own card, a Stop button, and a read-only transcript. This gives much better visibility when Claude Code fans out into several parallel workers on a larger refactor across your Blazor solution.

VS Code: Hooks & Permission rules dialogs

VS Code gained dedicated dialogs for viewing and editing hooks and permission rules across user, project, and local settings, without hand-editing JSON files. This makes it much faster to tighten or relax what Claude Code is allowed to do per repository, e.g. inside your Azure DevOps checkouts.

Output style switching on demand (/output-style)

/output-style [name] lists and switches Claude Code's response style at any time, including in headless, cloud, and Remote Control sessions. Useful for terser output while triaging a pipeline failure vs. more explanatory output while reviewing new Blazor components.

3. CI/CD, Automation & Self-Hosted Infrastructure

Self-hosted runner reliability & cleanup

The self-hosted runner (relevant to your on-prem setup) gained **--remove-session-state** to auto-clean per-session directories, **--drain-marker-file** for graceful shutdown reporting on SIGTERM, and a fix for sessions silently losing all host config when the config directory exceeds 64 MiB. Together these reduce disk growth and make long-running on-prem sessions more predictable.

Per-command allowed_domains in sandboxed auto mode

Bash, PowerShell and Monitor commands running under auto mode with sandboxing can now declare per-command **allowed_domains**, so only the hosts that specific command needs are opened for it and every other host is refused. This tightens network egress control for tasks that touch your on-prem Azure DevOps server or internal package feeds.

Managed MCP servers for the whole team

Organizations can push shared HTTP/SSE MCP servers to every developer via the new **managedMcpServers** setting, instead of everyone configuring their own **.mcp.json**. A good fit for rolling out shared internal tools (ticketing, internal APIs) across your 5–10 person team centrally.

Workflow tool: higher agent concurrency

CLAUDE_CODE_WORKFLOW_MAX_CONCURRENT_AGENTS (1–256) raises the Workflow tool's concurrent-agent limit for large, inference-bound fan-out jobs. Useful if you build a Claude Code workflow that processes many files or services in parallel across a larger .NET solution.

4. Model, Cost & Performance Control

Per-model effort levels

maxEffortLevel now caps the model's "thinking effort" per provider (including Bedrock/Vertex/Foundry), and **/effort** remembers a separate preferred effort level for each model. This gives you a direct lever to trade quality for cost/latency — light effort for routine boilerplate, higher effort for hard debugging.

Prompt-cache visibility

/cost and the status line now surface a likely cause for prompt-cache misses (changed tool definitions, idle past the cache TTL) along with hit ratio and tokens re-cached. This makes it much easier to understand why a long session suddenly got more expensive and to structure work to preserve the cache.

Claude Fable 5.1

Claude Fable 5.1 is now available with a 1M-token context window and cheaper cache reads, and is the new default Fable model where enabled. It's aimed at very large-context tasks, such as reasoning across a large legacy .NET codebase in a single pass.

5. Windows / .NET Environment Fixes

PowerShell long-path fix

A fix landed for PowerShell tool commands failing with "Exit code 1" and no output when the session's temp output path exceeded 260 characters — a common issue on deeply nested Windows repos. This removes a

real source of silent, hard-to-diagnose failures on Windows dev machines.

AppContainer / restricted-token sandbox fix

Read, Write and Edit tools were wrongly refusing every file when Claude Code ran inside a Windows AppContainer or a restricted-token sandbox. Both are common in locked-down enterprise .NET environments, so this removes friction if your machine or CI agents run under such a sandbox.

Source: Claude Code changelog (code.claude.com/docs/en/changelog), covering releases up to v2.1.272 (September 15, 2026). Claude Code updates very frequently — check the changelog directly for anything newer.